BACHELOR'S THESIS (TFG) · UC3M

Retrain without forgetting how to generalize

Warm starting, Shrink & Perturb, and what happened on CARLA · 5-minute talk (7 slides)

[Your name] · Supervisor: [name] · September 2026

Part 1 · The paper in depth (for you, not to read aloud)

Ash & Adams, *On Warm-Starting Neural Network Training*, NeurIPS 2020 (`papers/warm_start.pdf`).

The problem

Real systems receive data piece by piece and retrain each time. The obvious shortcut is to **warm start**: begin the new training from the previous model instead of from a fresh initialization. It is faster, and intuition says the old model is a good starting point. The paper shows that, for neural networks, it is not.

Evidence that warm starting hurts (section 2)

- **Two-step test:** train to convergence on 50% of the data, then on 100%. Compared with training on 100% from a random start, test accuracy drops for ResNets and MLPs: ResNet-18 on CIFAR-10 **56.2% → 51.7%** (SGD) and **78.0% → 74.4%** (Adam); on CIFAR-100 with Adam **41.4% → 35.0%** (Table 1).
- **Training loss is just as good.** The warm model fits the training data equally well; it is worse only on unseen data. That is a *generalization* problem, not an optimization one.
- **Logistic regression is not affected.** Its loss is convex, so there is only one solution to find. The problem is specific to non-convex neural networks.
- **Online test (Figure 2):** CIFAR-10 arrives in chunks of 1,000 images. Warm starting is much faster, but its accuracy gap to retraining from scratch **grows as more data arrives**.

The obvious fixes don't work (section 3)

- **Learning rate and batch size:** some settings close the gap, but those runs take as long as training from scratch, so the speed advantage is gone (Figure 3).
- **Stopping round 1 early** doesn't help: even a few epochs on the first chunk already damage the final model (Figure 4).
- **Regularization** (L2 weight penalty, confidence penalty, adversarial training) helps both, but warm-started models stay behind (section 3.3).

Why it happens (Figure 5)

After round 1 the model already fits the old data almost perfectly, so the old data produce **tiny gradients** and the new data produce **large ones**. Training is then driven by the new data alone, and the model ends in a solution that fits the old data by memory rather than by general patterns. Randomly initialized models get similar gradients from all data.

The fix: Shrink & Perturb (section 4)

Before each new round: $\theta \leftarrow \lambda\theta + \text{noise}$, with $0 < \lambda < 1$.

- **Shrink** (multiply all weights by λ, e.g. 0.6): the model keeps its learned patterns but becomes less confident. Its loss rises on **all** data, so old and new data give **balanced gradients** again (Figure 5).

- **Perturb** (add a small, scaled-down freshly initialized network, e.g. 0.01×): helps the model train faster afterwards.
- **Result:** with $\lambda = 0.6$, noise 0.01 it matched training from scratch in accuracy while training much faster; stronger shrinking even beat it (Figures 7–8).

Caveat: The paper tested image **classification** only, with **no data augmentation and no learning-rate schedule** (section 3). YOLO uses both. Whether the effect survives that is exactly what my experiment tests.

Part 2 · My experiment in depth

- **Data:** the previous thesis's Roboflow dataset (v1, no augmentation): 3 flight days (17 Nov, 5 Dec, 11 Dec 2025), video frames cut into 640 px tiles. I removed 72 duplicated tiles from train, 3 of which leaked into valid/test. Train 1864 tiles, valid 704, test 696.
- **Simulating new flights:** round 1 = 17 Nov; round 2 adds 5 Dec; round 3 adds 11 Dec. Valid and test stay fixed.
- **Three ways to start each round: cold** (COCO-pretrained weights), **warm** (last round's final weights), **Shrink & Perturb** (last round's weights, $\lambda = 0.6$, noise 0.01, BatchNorm statistics untouched).
- **Same everything else:** YOLO26n, 30 epochs per round (the original training curve reaches 98% of its best by epoch 30), batch 16, `box=5.0` as in the original thesis, 3 seeds, Tesla T4 on Colab. Round 1 is identical for all three and is trained once per seed.

Results (test mAP50 of the final model, mean ± std over 3 seeds)

Round	Cold	Warm	Shrink & Perturb	Train time
1	0.768 ± 0.005	same run	same run	10.5 min
2	0.796 ± 0.004	0.793 ± 0.005	0.799 ± 0.002	≈ 18.5 min
3	0.821 ± 0.003	0.815 ± 0.003	**0.829 ± 0.002**	≈ 21 min

- **Same order as the paper:** by round 3 warm is 0.6 points below cold; Shrink & Perturb is 1.4 above warm and 0.8 above cold.
- **The gap grows** from round 2 to round 3, as in the paper's Figure 2.
- **Free:** same training time for all three.
- **Smaller than in the paper** (about 1 point of mAP50 vs. 4–6 accuracy points). My explanation, not tested: YOLO's augmentation and per-run warmup already counter part of the problem.
- **Limits:** one dataset, 3 rounds, 3 seeds, one λ. Cold vs. warm alone is borderline with 3 seeds; Shrink & Perturb vs. warm is clear.

Part 3 · Script

#	Slide	Time
1	Every flight means a retrain	0:20
2	The shortcut: warm starting	0:40
3	Why it hurts	0:45
4	Shrink & Perturb	0:45
5	My experiment	0:50
6	Results	0:50
7	What I take from it	0:30

About 630 words: roughly 4:30 at 140 words per minute, leaving time for pauses and slide changes. If running late, shorten slide 3.

Slide 1 · Every flight means a retrain

0:00 – 0:20

On screen: title; drone photo of the field

> My thesis builds a cardilla detector that keeps improving as new drone flights arrive. Every new flight means retraining the model. Today I'll talk about one practical question behind that: when we retrain, where should training start from?

Slide 2 · The shortcut: warm starting

0:20 – 1:00

On screen: three boxes: round 1 → round 2 → round 3, each arrow labeled "start from previous model?"

> The obvious answer is to continue from the model we already have. That's called warm starting. It's cheap, and it feels right: the old model already knows what a cardilla looks like. Ash and Adams, in a NeurIPS 2020 paper, showed that for neural networks this intuition is wrong. If you train a ResNet on half of CIFAR-10 and then continue on all of it, it ends up clearly worse on new images than one trained on all the data from the start: 52% instead of 56% accuracy. And the more rounds you do, the bigger the gap gets.

Slide 3 · Why it hurts

1:00 – 1:45

On screen: two bars: gradient size from old data (tiny) vs. new data (large)

> The surprising part is that the warm model fits the training data just as well. It only fails on images it hasn't seen. So it's not that it can't learn; it learns the wrong way. After round one, the old data are already fitted perfectly, so they produce almost no gradient. Training is driven only by the new data, and the model ends up memorizing instead of generalizing. The authors tried the obvious fixes: other learning rates, batch sizes, regularization. They either don't close the gap, or they close it only by making training as slow as starting over.

Slide 4 · Shrink & Perturb

1:45 – 2:30

On screen: formula: new weights = 0.6 × old weights + 0.01 × random weights

> Their fix is very simple. Before each new round, multiply all weights by a number below one, for example 0.6, and add a little random noise. Shrinking keeps what the model learned but makes it less confident, so old and new data both produce useful gradients again. The noise helps it train faster. In their experiments this matched training from scratch, while keeping most of the speed of warm starting. But one caveat: they only tested image classification, without data augmentation. Detectors like YOLO use both, so it wasn't clear the problem even exists there.

Slide 5 · My experiment **2:30 – 3:20**

On screen: timeline: 17 Nov → 5 Dec → 11 Dec, cumulative; three starting points: cold / warm / S&P

> So I tested it on our data. We have labeled tiles from three flight days. I replay them in order, as if each day were a new flight: round one trains on the first day, round two adds the second, round three adds the third. In each round I compare three starting points: cold, meaning the standard COCO-pretrained weights; warm, meaning last round's model; and Shrink and Perturb. Everything else is identical: YOLO26 nano, thirty epochs per round, the same fixed test set, three seeds each. Before that, I also cleaned the dataset: some frames were uploaded twice, and three of them were in both training and test.

Slide 6 · Results **3:20 – 4:10**

On screen: results table, round 3 row highlighted

> Round one is the same for everyone. By round three, the order is exactly what the paper predicts. Warm starting is the worst, at 0.815 mAP50. Retraining from COCO gets 0.821. And Shrink and Perturb gets 0.829, better than both, at the same training time. The gap also grows from round two to round three. The effect is smaller than in the paper, about one point instead of four to six, probably because YOLO's augmentation already counters part of the problem. But it is consistent across seeds and costs nothing.

Slide 7 · What I take from it **4:10 – 4:40**

On screen: three bullet points; "next: TAILOR"

> Three takeaways. First, warm starting does hurt a detector, a little, and the damage builds up with every round. Second, Shrink and Perturb fixes it for free, about fifteen lines of code. Third, this decides how I retrain in the next step of my thesis: a reinforcement-learning bandit that chooses which tiles to label after each flight. That loop has many more rounds, so getting retraining right matters even more there. Thank you.

Likely questions

- **Why not just always retrain from COCO?** You can, at the same cost here, but Shrink & Perturb was better and keeps what the model learned. With many more rounds, warm-based retraining can also use fewer epochs.
- **Is one point of mAP50 meaningful?** It is several times larger than the seed-to-seed variation in round 3, and the gain is free. It's about a third of what adding a whole flight day of labels gave.
- **Why λ = 0.6?** It is the paper's main setting. I didn't tune it; testing 0.3 and 0.9 is planned.
- **Why 30 epochs and not 200 like the original thesis?** The original training curve reaches about 98% of its best validation mAP50 by epoch 30 and overfits after about 50. All setups use the same number, so the comparison is fair.
- **Would the gap keep growing with more flights?** Probably against warm starting (the paper and my 2-round trend both point that way). Against retraining from scratch it's unknown; splitting the data by video (10 rounds) would test it.